BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 009 · Sequence & branching in C#

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. Why would Unicode be chosen instead of standard ASCII for a multilingual application?

Q2. Convert 01011000 0011 to denary using an 8-bit two’s-complement mantissa and a 4-bit two’s-complement exponent.

Q3. Convert $6D_{16}$ to eight-bit binary.

Q4. Convert 11110110 from 8-bit two’s-complement binary to denary. Show signed place values.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

Recall the term — Sequence

Recall the term — Branching / selection

Recall the term — Boolean expression

Recall the term — Condition

Recall the term — Relational operators

Recall the term — Logical operators

Recall the term — Parsing

New learning

Sequence = instructions in order — Sequence is the normal top-to-bottom execution of statements. Each statement happens after the previous one unless another construct changes the flow. — CODE:

```
Console.Write("Name: ");
string name = Console.ReadLine();
Console.WriteLine("Hello " + name);
```

Input arrives as text — Console.ReadLine() returns a string. If the program needs a whole number, parse the input into an int . — CODE:

```
Console.Write("Age: ");
int age = int.Parse(Console.ReadLine());
```

Branching chooses a route — An if condition must evaluate to true or false . Only the matching branch is executed. — CODE:

```
if (age >= 18)
{
Console.WriteLine("Adult");
}
else
{
Console.WriteLine("Under 18");
}
```

Conditions and order matter — Use == to compare equality, not = . Use && , || and ! for AND, OR and NOT. — With else if , conditions are tested from top to bottom. Put the most demanding threshold first.

Worked example

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Build a result checker

1. Use the standard .NET Fiddle structure — CODE:
using System;

public class Program
{
public static void Main()
{
// code goes here
}
} — Main program instructions belong inside Main() .

2. Read the score as an integer — CODE:
Console.Write("Enter score: ");
int score =
int.Parse(Console.ReadLine()); — Console.ReadLine() gives us text; int.Parse(...) converts it to a whole number.

3. Test the highest threshold first — CODE:
if (score >= 70)
{
Console.WriteLine("Distinction");
}
else if (score >= 50)
{
Console.WriteLine("Pass");
}
else
{
Console.WriteLine("Not yet");
} — A score of 85 satisfies both >= 70 and >= 50 , so the higher threshold must be tested first.

4. Use values that expose mistakes — CODE:
85 → Distinction
70 → Distinction
50 → Pass
49 → Not yet

We Do · Practise with guidance

Even or odd

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Ask for a whole number. Output Even when the remainder after division by 2 is zero; otherwise output Odd .

Build the missing decision

```
Console.Write("Number: ");
int number = int.Parse(Console.ReadLine());

// ?
```

```
Console.Write("Number: ");
int number = int.Parse(Console.ReadLine());

if (number % 2 == 0)
{
    Console.WriteLine("Even");
}
else
{
    Console.WriteLine("Odd");
}
```

C# % is the remainder operator; it matches OCR MOD .

Three temperature bands

Output Hot for 25+, Mild for 15–24, otherwise Cold .

Which condition comes first?

Test the highest threshold first so 25+ is not caught by the 15+ branch.

Age range using AND

Output Teen ticket only when the age is from 12 to 17 inclusive.

Write one Boolean condition

if (age >= 12 && age <= 17)

Name it and show it

A past OCR programming-techniques question expects the three constructs to be named and supported with a working example .

Today: give a C# example of sequence and a C# example of branching . Iteration comes next lesson.

Use the technical vocabulary: sequence, branching/selection, iteration.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Positive, negative or zero
Ask the user for an integer. Output exactly one of:
Positive · Negative · Zero

Question 2

Ticket category
Ask for an age and output:
Child for 0–11 · Teen for 12–17 · Adult for 18+

Question 3

Delivery charge
Ask for an order total and whether the customer is a member.
Output Free delivery only when the total is at least 30 AND the member answer is "yes" . Otherwise output Delivery charge applies .

Question 4

Simple login
Ask for a username and a PIN.
Output Access granted only when the username is "admin" and the PIN is 2468 .
Otherwise output Access denied .

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Exit Ticket · Answer before checking

In C#, == is used to compare two values for equality.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. Unicode supports a much larger repertoire of characters from many writing systems. Standard ASCII has 128 codes and is centred on a limited character set. Unicode provides code points for many languages and symbols.

Q2. 5.5 Mantissa 01011000 = 0.6875 ; exponent 0011 = 3 ; $0.6875 \times 2^3 = 5.5$.

Q3. 01101101_2 6 → 0110 and D → 1101 .

Q4. −10 $-128 + 64 + 32 + 16 + 4 + 2 = -10$.

You Do pages · Independent questions

Q1. Positive, negative or zero
CODE:

```
using System;

public class Program
{
public static void Main()
{
Console.Write("Enter an integer: ");
int number = int.Parse(Console.ReadLine());

if (number > 0)
{
Console.WriteLine("Positive");
}
else if (number < 0)
{
Console.WriteLine("Negative");
}
else
{
Console.WriteLine("Zero");
}
}
```

```
}
```

Why it works: the first two conditions remove positive and negative values; the only remaining possibility is zero.

Q2. Ticket category

CODE:

```
using System;

public class Program
{
public static void Main()
{
Console.Write("Enter age: ");
int age = int.Parse(Console.ReadLine());

if (age <= 11)
{
Console.WriteLine("Child");
}
else if (age <= 17)
{
Console.WriteLine("Teen");
}
else
{
Console.WriteLine("Adult");
}
}
}
```

Why the order works: once ages 11 and below have been removed, age <= 17 means 12–17.

Q3. Delivery charge

CODE:

```
using System;

public class Program
{
public static void Main()
```

```
{
Console.Write("Order total: ");
double total = double.Parse(Console.ReadLine());

Console.Write("Member? yes/no: ");
string member = Console.ReadLine();

if (total >= 30 && member == "yes")
{
Console.WriteLine("Free delivery");
}
else
{
Console.WriteLine("Delivery charge applies");
}
}
}
```

Key idea: && means both Boolean expressions must be true.

Q4. Simple login

CODE:

```
using System;

public class Program
{
public static void Main()
{
Console.Write("Username: ");
string username = Console.ReadLine();

Console.Write("PIN: ");
int pin = int.Parse(Console.ReadLine());

if (username == "admin" && pin == 2468)
{
Console.WriteLine("Access granted");
}
else
{
```

```
Console.WriteLine("Access denied");
}
}
}
```

Key idea: equality uses == . The login succeeds only when both comparisons are true.

Exit Ticket · Model answer

True. In C#, == tests whether values are equal; = is assignment.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed: